

COS30017 - Software Development for Mobile Devices

Assignment 1: Gymnastics App

Nguyễn Chí Cường - 104852250

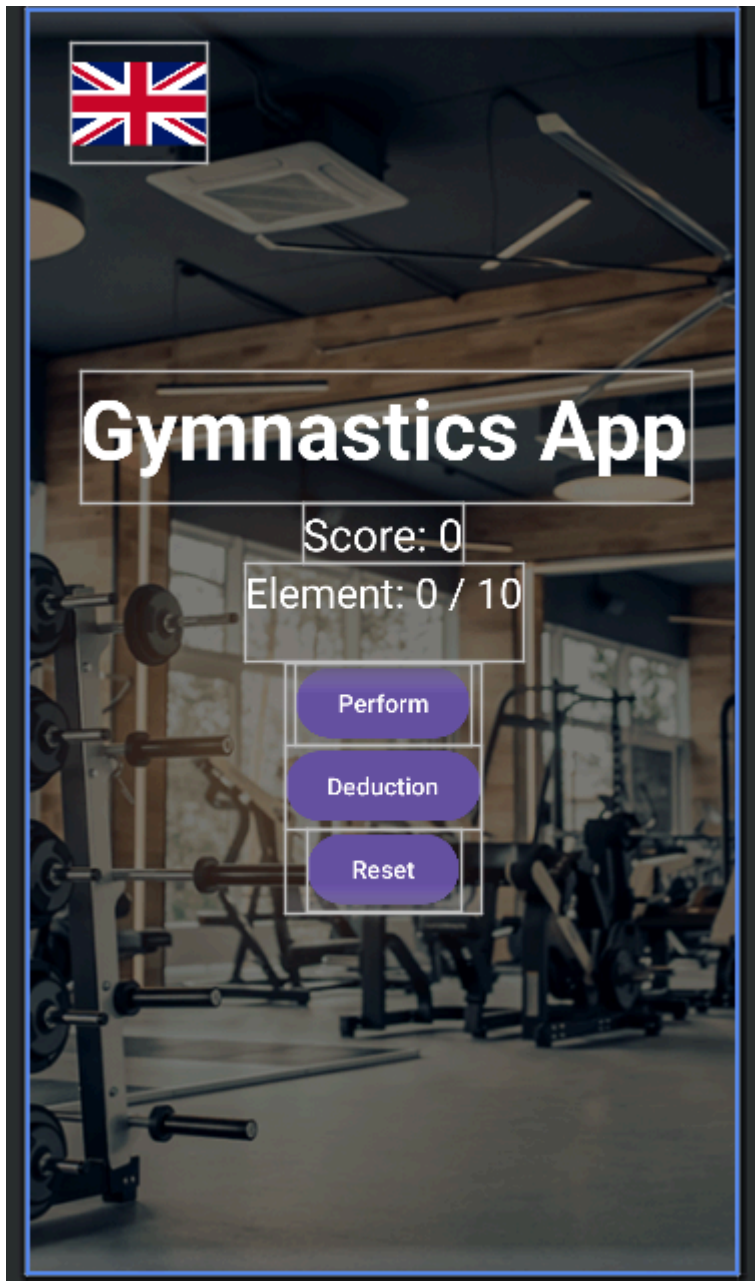


Git repository: <https://github.com/Opatzo2212/COS30017.git>

1. My development plan

During this assignment, I have decided to do it over a 5 day period. Each day is given a different task in order to be as efficient as possible.

Day 1: I will create the basic interface of the app.



How the app look

Day 2: I will create the basic functionality of the app which is the three buttons: Perform, Deduction and Reset. Perform will be able to increase score(by 1 for now), deduction will reduce the score(by 2) and reset will reset the score to 0.

```
private fun Perform() {
    if (currentElement < 10 && !hasTakenDeduction) {
        currentElement++

        val points = when (currentElement) {
            in 1 ≤ .. ≤ 3 -> 1
            in 4 ≤ .. ≤ 7 -> 2
            in 8 ≤ .. ≤ 10 -> 3
            else -> 0
        }

        currentScore += points
        Log.d( tag = "Debug", msg = "Score changed")
        updateUI()
        if (currentElement == 10) {
            val completionMsg = if (isVietnamese) "Buổi tập đã hoàn thành!" else "Routine Complete!"
            Toast.makeText( context = this, text = completionMsg, duration = Toast.LENGTH_LONG).show()
        }
    } else if (hasTakenDeduction) {
        Toast.makeText( context = this, text = "Cannot perform after deduction!", duration = Toast.LENGTH_SHORT).show()
    }
}
```

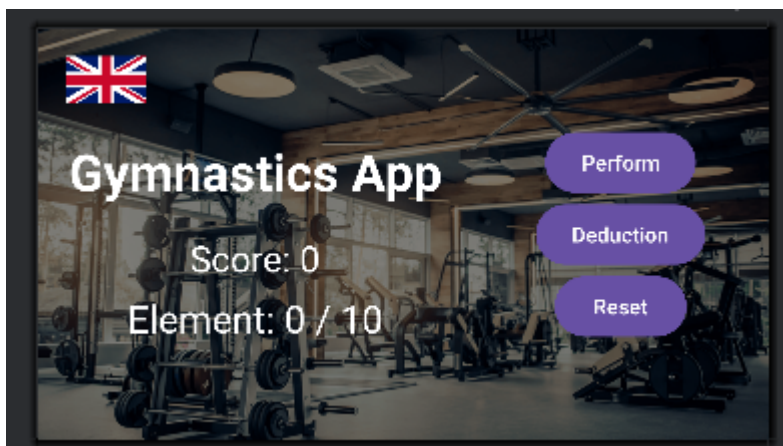
Example code for perform function

Day 3: I will add the more intricate function of the app including the 10 elements routine, the three difficulty zones and other logic.

```
val points = when (currentElement) {
    in 1 ≤ .. ≤ 3 -> 1
    in 4 ≤ .. ≤ 7 -> 2
    in 8 ≤ .. ≤ 10 -> 3
    else -> 0
}
```

The difficulty zone code

Day 4: I will add the two required layouts and savedInstanceState to save the score on rotation.



Second layout for landscape

Day 5: I will add a language change option.

```
@SuppressWarnings("SetTextI18n")
private fun updateUI() {
    if (isVietnamese) {
        title.text = "Ứng dụng thể dục"
        score.text = "Điểm: $currentScore"
        cur_el_num.text = "Động tác: $currentElement"
        btnPerform.text = "Thực hiện"
        btnDeduction.text = "Trừ điểm"
        btnReset.text = "Đặt lại"
        btnLangague.setImageResource(R.drawable.vn_flag)
    } else {
        title.text = "Gymnastics App"
        score.text = "Score: $currentScore"
        cur_el_num.text = "Element: $currentElement"
        btnPerform.text = "Perform"
        btnDeduction.text = "Deduction"
        btnReset.text = "Reset"
        btnLangague.setImageResource(R.drawable.flag_en)
    }
}
```

2. Key decisions made

- **Model-View Architecture:** I separated the scoring logic into a helper method to ensure that updating the UI always reflected the current state of the "Routine" (the list of 10 elements), rather than just incrementing a raw integer.
- **State Persistence:** Using Bundle via onSaveInstanceState was a critical decision to ensure the user doesn't lose a half-completed gymnastics routine when the device orientation changes.
- **Difficulty Scaling:** I used distinct buttons for Difficulty Zones to ensure only one zone could be active per "Perform" click, preventing input errors.

3. Issues encountered

Issue Encountered	Solution
-------------------	----------

Data Loss on Rotation: The current score and routine count reset to zero whenever the screen was rotated.	Implemented onSaveInstanceState to store the double value of the score and the integer count of elements.
Landscape Overflow: The "Reset" button was pushed off-screen in landscape mode due to the height of the element list.	Created a dedicated landscape layout that repositioned the buttons to the side of the score display.

4. Reflection

What Worked Well

- Incremental Testing: Building the core "Perform/Deduction" logic before adding complexity (zones/limits) allowed me to identify bugs in the scoring math early on.
- Clear Tasks Management: By separating tasks for each day I could get the work done better and easier than simply doing it without any planning beforehand.

What Could Be Improved

- Code Efficiency: While the logic is functional, I could have used a more robust MVVM (Model-View-ViewModel) pattern. This would have handled the state persistence more elegantly than manually saving to a Bundle.
- UI Feedback: I would like to add visual animations (e.g., a flash or a sound) when the 10-element limit is reached to give the user better immediate feedback that no more scores can be added.